

SpeakForge Local Audit Report

Audit date: 2026-08-21

Executive summary

SpeakForge is a FastAPI and React application for authenticated speaking and writing tests. The local MySQL service is available, the speakforge database is reachable with the configured local credentials, the required tables exist, and the frontend production build succeeds.

Verified local configuration

- Database engine: MySQL through PyMySQL.
- Database name: ``speakforge``.
- Database host and port: ``localhost:3306``.
- Database user: ``root``.
- Password: configured in the ignored root ``.env``; it is intentionally not reproduced in this report.
- Backend environment: ``backend\venv``.
- Backend command: ``cd backend` then `uvicorn app.main:app --reload --host 127.0.0.1 --port 8000``.
- Frontend command: ``cd frontend` then `npm run dev``.

Architecture and pipeline

1. React routes render user, admin, authentication, test, and report pages.
2. Frontend requests call the FastAPI service at 127.0.0.1:8000.
3. FastAPI authenticates users with JWT and applies role checks to admin operations.
4. SQLAlchemy persists users, tests, questions, answers, attempts, and results in MySQL.
5. Text answers are evaluated directly; audio answers are uploaded, transcribed with Whisper, and evaluated.
6. Grammar and fluency scores are stored on question answers.
7. The report endpoint aggregates completed answers and returns overall score, averages, level, and per-answer feedback.

Implemented feature inventory

- Registration, login, JWT authentication, email verification, and password reset flows.
- Admin test creation, editing, deletion, question management, and analytics views.
- User dashboard, available tests, test detail, answer submission, and result history.
- Written-answer and audio-answer submission paths.
- AI scoring service integration for transcription, grammar analysis, and scoring.
- Protected routes and role-based access control.
- Responsive React interface with Tailwind CSS, charts, alerts, and loading states.

Database schema verified

The following tables were returned by SQLAlchemy inspection in the active local database: users, tests, questions, question_answers, attempts, and results.

Corrections made during audit

- Environment loading now resolves the repository ``.env`` from the backend code location, independent of the current working directory.
- The duplicate report implementation was removed so the application has one report contract.
- The report router is mounted at `/tests/report/{test_id}``, matching the frontend request.
- The permissive wildcard CORS origin was removed while retaining the two local Vite origins.
- ESLint configuration was aligned with the installed ESLint 8 toolchain.
- README database and report endpoint documentation now match the local MySQL setup and running code.

Validation results

- Backend import: passed with ``backend\venv``.
- MySQL connection: passed; ``SELECT 1`` returned successfully.
- Active database probe: passed; returned ``speakforge``.
- Schema inspection: passed; six expected tables found.
- Frontend build: passed.
- Frontend lint: validated after the final compatibility adjustment.

Known limitations and operational risks

- AI scoring depends on external/runtime model assets and LanguageTool availability; first startup may download or initialize large assets.
- Email verification and password reset require SMTP environment variables; without them, email delivery is skipped.
- The local ``.env`` contains a simple development password and must not be reused outside local development.
- The frontend bundle currently produces a chunk larger than Vite's 500 kB advisory threshold; this is a performance warning, not a build failure.
- No automated backend integration test suite was present in the repository, so endpoint behavior beyond startup and database checks should be covered next.